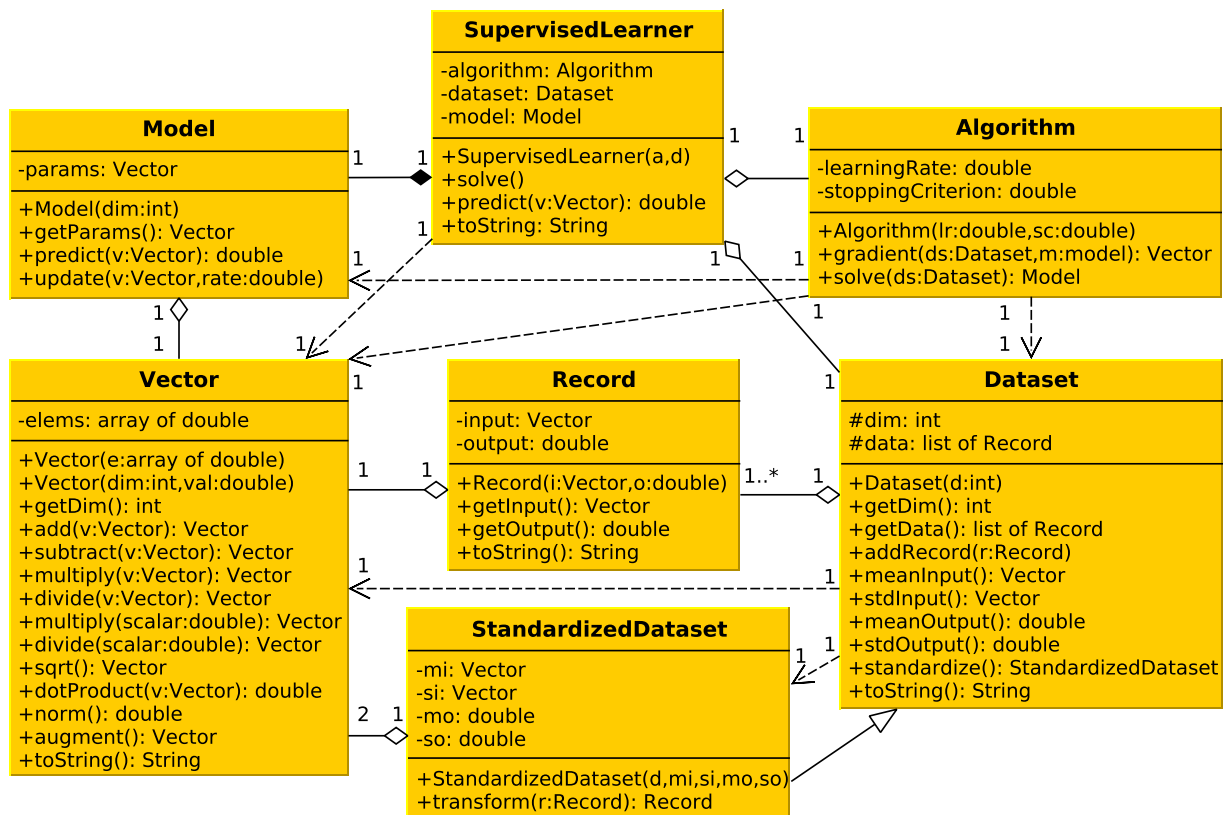


Lab 3: Implementing inheritance

24292-Object Oriented Programming

1 Introduction

The aim of this lab is to implement the design of Seminar 3. The session is mandatory and you have to deliver the source code of the Java project as well as a document that outlines the solution of the problem, using the guidelines from Lab 1. As usual, a sample solution appears below.



As before, define a `TestLearner` class with a main method where the other classes are tested. Frequently test the classes in the main method by creating instances and calling methods on them.

2 Existing classes

Start by incorporating the classes from Lab 2, copying the Java files into a new directory. The classes `Vector`, `Record`, `Dataset` and `StandardizedDataset` are practically the same as before, except that the `Vector` class needs a new method `augment` to compute and return an augmented vector as described in the handout of Seminar 3.

3 The Model class

The `Model` class should represent a $(d + 1)$ -dimensional parameter vector θ , where d is the dimensionality of the inputs of the associated dataset. It is convenient to use the `Vector` class to represent θ . Apart from the constructor and the getter method, the `Model` class needs two methods `predict` and `update` which are both described in Seminar 3. Both methods can be implemented using the existing methods of the `Vector` class.

4 The Algorithm class

The `Algorithm` class is responsible for implementing gradient descent. The relation between `Algorithm` and the classes `Vector`, `Dataset` and `Model` is of type use/dependency since `Algorithm` has no attributes involving these classes, only methods in which these classes are referenced.

The `Algorithm` class has an attribute representing the learning rate α , which is usually small (in the range $[10^{-4}, 10^{-1}]$). A larger learning rate makes the model converge faster, but setting the learning rate too large can cause gradient descent to overshoot the target and never reach the optimum.

In addition, to decide when to stop iterating we can also include an attribute representing a stopping criterion. Concretely, if the learning rate is appropriately selected, each iteration brings the model closer to the optimum, and as a consequence, the norm of the gradient becomes smaller and smaller. We can set the stopping criterion to a small value (e.g. 10^{-6}) and stop iterating when the norm of the gradient becomes smaller than this value.

The method `gradient` should compute the gradient of the loss function according to Equation (2) in the handout of Seminar 3. The method `solve` is where gradient descent is implemented. This method needs to initialize a model to a default value and iteratively compute the gradient vector using the method `gradient` and call the method `update` of `Model` until the norm of the gradient is smaller than the stopping criterion.

5 The SupervisedLearner class

The SupervisedLearner class manages all the components involved in linear regression: the dataset, the model and the algorithm. The class has a composition relation with Model since it controls the creation of Model instances, but an aggregation with Dataset and Algorithm since those objects can exist outside SupervisedLearner. In addition there is a use/dependency relation with Vector since the method `predict` has an argument of type Vector.

Apart from the constructor method, the method `solve` should delegate solving to the instance of Algorithm and store the resulting model in its attribute. The method `predict` takes an input vector as argument and returns the predicted output given the current model. Note that the input vector is not augmented in this case, and has to be augmented to make the prediction. The intention of the `toString` method is to convert the current model to string format in order to print the parameters to the screen.

6 Optional: Moore-Penrose inverse

For extra credit, implement a solution for linear regression based on the Moore-Penrose inverse. To do so, you first have to create an $n \times (d + 1)$ data matrix X whose rows are the augmented input vectors of the dataset, and an n -dimensional output vector y whose elements are the outputs of the dataset. It is important that the inputs and outputs are ordered in the same way in X and y . The optimal parameter vector is given by

$$\theta^* = X^\dagger y,$$

where $X^\dagger$ is the Moore-Penrose inverse of the matrix X which has dimensionality $(d + 1) \times n$. The task is to compute the parameter vector this way and compare it to the solution obtained using gradient descent.

It is difficult to implement code for the Moore-Penrose inverse from scratch. Instead, a better idea is to use an existing Java library for computing the inverse, for example one of the following two:

https://ejml.org/javadoc/org/ejml/dense/row/linsol/svd/SolvePseudoInverseSvd_DDRM.html

<https://commons.apache.org/proper/commons-math/javadocs/api-3.6.1/org/apache/commons/math3/linear/SingularValueDecomposition.html>

To use these libraries, it is necessary to represent matrices and vectors using the classes of the library, which in turn requires reading the documentation.

7 Submission

The source code and documentation should all be uploaded to a directory **Lab3** of your Git repository **prior to the next lab**. Please make sure that the directory name is *exactly* **Lab3** (not **Lab_3**, **Lab-3**, **LAB3**, **lab3** etc.), and that **Lab3** is located in the **root of the Git repository**. Failure to comply with these rules will result in a grade deduction.